Movement Quest

Offline workout logbook

Save workout plans, record reps and weights,
and review workout history on the phone.

Mario Leonel Gutierrez Espinal
Team Fitness (solo)

Project scope

I changed my original Trail Tales proposal because route tracking and safety warnings did not fit the assignment.

Movement Quest records workout information and stores it on the phone. It does not give health or exercise advice.

Area	Questions
Required	I1, I2, D1, D2, R4
Architecture	A1, A2
Local data	D4, D5
Testing	T1, T2
Reflection	R5

Widgets and state

WorkoutCard (stateless)

Displays one plan: title, equipment, effort and note.

SetRow (stateless)

Displays a saved set. Workout and History use the same widget.

WorkoutScreen (stateful)

Tracks the session, selected exercise, logged sets and busy flag. Input stays in memory until a successful save writes the set to SQLite.

How a set is saved

Part	Responsibility
WorkoutScreen	Collects input and displays results
validators.dart	Checks reps and weight
TrainingLog	Converts the set to a map
DatabaseHelper	Writes the row to SQLite

Keeping these jobs separate avoids repeated SQL in screens and makes the validation and database code easier to test.

Leaving the screen during a save

dispose() releases the reps and weight text controllers when the screen closes.

After the database call:

```
await DatabaseHelper.instance.insertLog(draft);
if (!mounted) return;
setState(() { ... });
```

The save can finish after the screen closes. The mounted check prevents an update to a disposed screen.

There is no automated test for navigating away while a save is running.

Local storage

SQLite (sqflite)

Stores workout_plans and training_logs. Both tables support creating, reading, updating and deleting records.

SharedPreferences

Stores default effort, preferred equipment, dark mode and the Gear Match setting.

The app works offline. Its release manifest requests no INTERNET permission.

Adding the weight column

Version 1

Saved sets have reps, but there is no weight column.

Version 2

onUpgrade adds the optional weight column with ALTER TABLE. Existing sets stay in the database; their weight starts as null.

weight_test.dart creates a version 1 database, opens it with the new code and checks that its saved set is still there.

Gear Match rules

Rule	Points	Skip when
Equipment matches the preference	+2	No preference is set
Plan differs from the last finished plan	+1	No workout has finished

Inputs: preferred equipment, saved plans and the last finished plan. The baseline compares equipment only.

Ties use the lower plan ID. View plan opens the plan without starting a workout. Not now hides the suggestion; Settings can turn it off.

Gear Match test cases

Case	Setup	Expected result	Pass
Normal	None preferred; B finished last	A, score 3	Yes
No plans	Empty plan list	No suggestion	Yes
No history	Mat preferred; no finished workout	Mat; skip +1	Yes
Tie	Plans 7 and 4; both Dumbbells	Plan 4	Yes
Known limit	Dumbbells saved; user wants Mat	Dumbbells wins	Yes

The rules use the saved preference, so they cannot tell what equipment the user wants today. AI_FEATURE.md documents this limit.

Tests and error handling

Test file	Tests	Coverage
acceptance_test	5	Counts, cascade, reset, reopen, demo
weight_test	2	Save, edit, clear, version 1 upgrade
gear_match_test	5	Gear Match cases
validators_test	5	Reps and weight validation
app_settings_test	3	Settings save and load

20 tests pass. Database tests use real SQLite files through sqflite_common_ffi.

Busy buttons block repeated actions. Invalid input shows a form error. Save failures show a retry message and keep the input.

Next improvement

Show the previous weight

When I start an exercise, I would like to see the weight and reps from the last workout.

For example: "Last time: 135 lb for 5 reps."

I would read this from the saved workout history and add a test with two sessions.